Model-Driven Software Development

Internal DSL Experimentation

Miguel Campusano

SDU Software Engineering

Previous class

Meta modeling

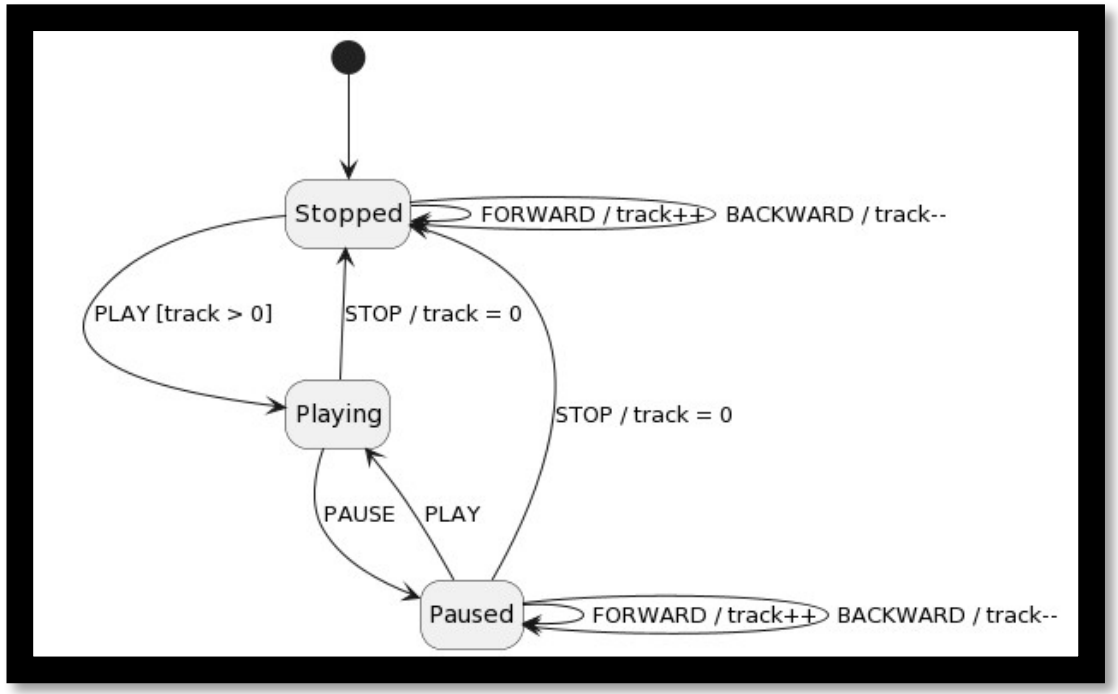
Summary

Models and modeling

Our first example: class diagrams

Warm-up Internal DSL

```
@Override
protected void build() {
      entity("Person").
        attribute("name", "String").
        attribute("age", "Number").
      entity("Student").
        attribute("id", "Number").
      entity("Teacher").
      entity("Course").
        attribute("title", "String").
      inheritance("Person", "Student").
      relation("Inheritance", "Person", "Teacher").
      relation("1-1", "Course", "Teacher").
      relation("Many-Many", "Course", "Student") ;
}
```

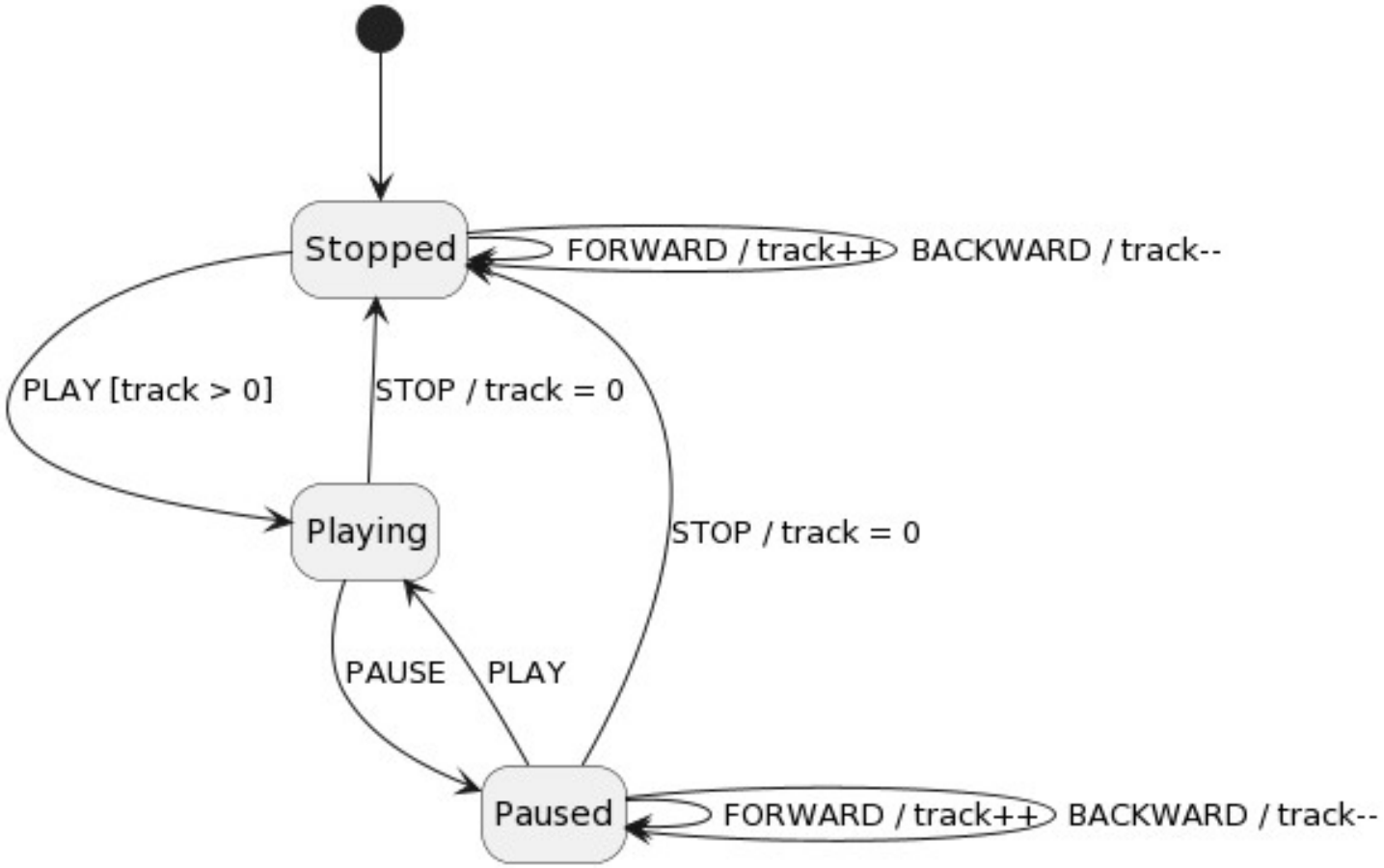


Model-driven approach

- Models are part of the development process
  - They are equal to code
  - We can *generate* code from models
    - Models are part of the software system

- Domain is essential for model-driven system
  - Domain drives abstractions (models)

State Machines: CD Player

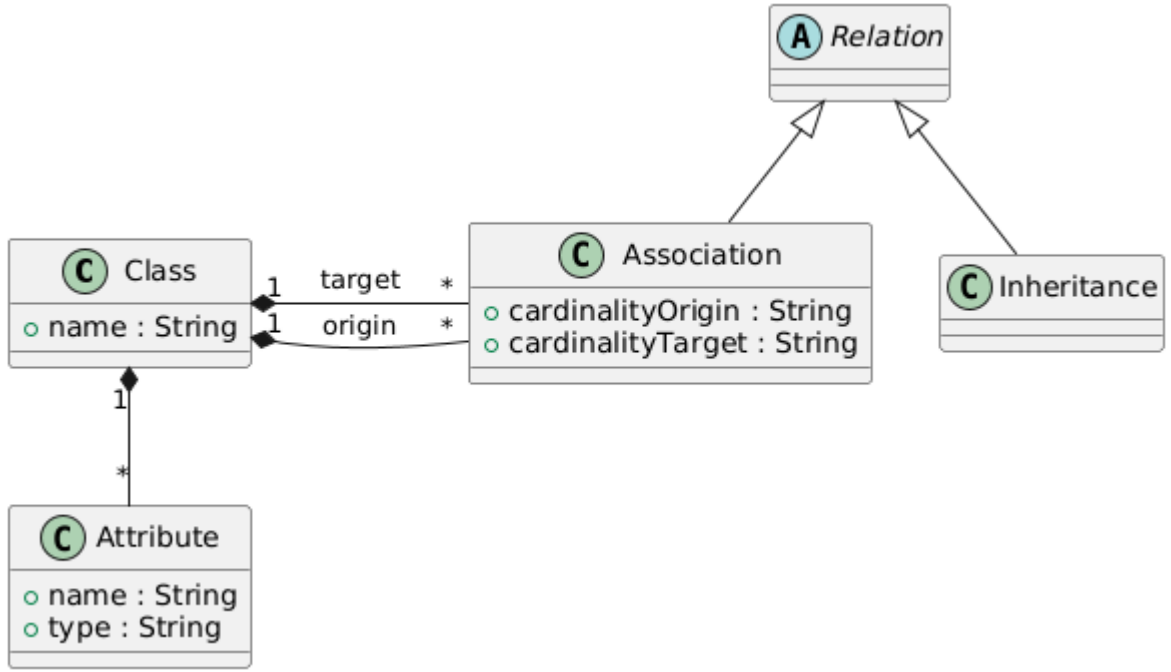


Metamodeling

Class Diagram

Rule of thumb

- 2 to 3 representative examples
- Abstract common properties
- Design (meta)model of examples
- Design DSL syntax



Internal DSL

- Host language generates a model
- Advantages of internal DSLs
  - Quick to implement, natural evolution
  - No additional tools, languages, infrastructure
- Disadvantages of internal DSLs
  - Model-based API can be cumbersome and confusing
  - Lack of static checking, IDE support
  - Mix of host language and DSL enables abuse

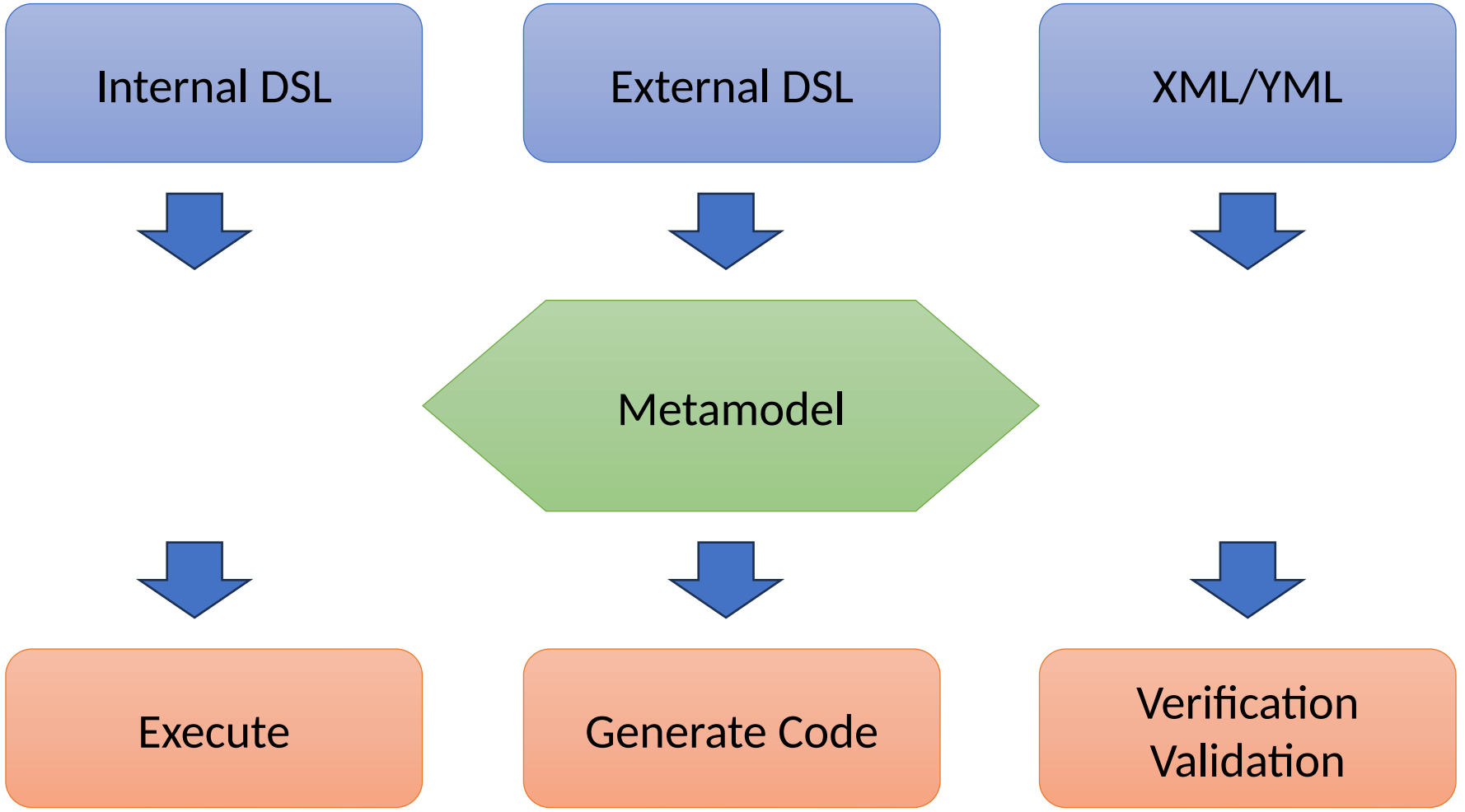
Discussion

Discussion: Metamodel & Internal DSLs

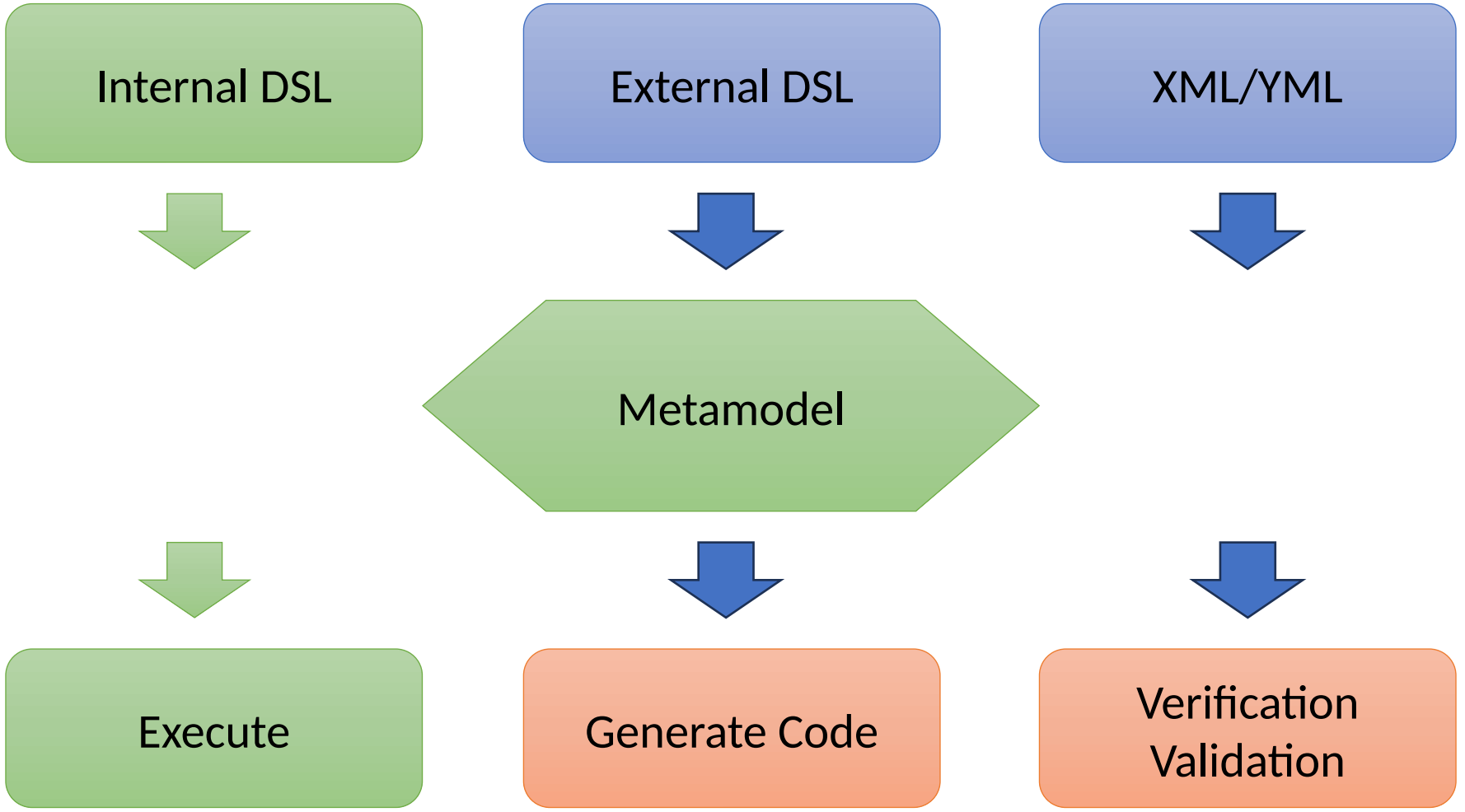
- Metamodel & Internal DSLs:
  - Is the concept of metamodel clear?
  - How are they related?
- Metamodel & Project
  - Too easy / too hard?
  - Too many things to model? How are you scoping your project?
  - Examples and syntax, how is it going?

PollEv.com/mica

Metamodel


Internal DSL
External DSL
XML/YML
Metamodel
Execute
Generate Code
Verification
Validation

Metamodel: Internal DSL interpretation



Model-Driven Software Development

Internal DSL Experimentation

Miguel Campusano

SDU Software Engineering

Today

- Warm-up Internal DSL
- Internal DSL Syntax
- Internal DSL Patterns

Internal DSL

An introduction

Internal DSLs

- DSL embedded into the syntax and semantics of a host language
  - Advantages: simple implementation, inherits advantages of host language
  - Disadvantages: syntax limitation, inherits disadvantages of host language
- Host-language dependent
  - OOP languages (Java, C#, Python): OOP *fluent interface*
  - Developers culture (Ruby, Smalltalk, etc): *business* as usual
  - C : Macros
  - ...

Internal DSLs: techniques

- Abstraction through metamodel
- Interpretation more than compilation
- Convenient syntax through patterns and *cleverness*
  - Method builder / factory
  - Method chaining
  - Macros
  - …

Internal DSL Syntax

Non-fluent API

```
@Override
protected void build() {
    Entity person = new Entity("Person");
    person.addAttribute(new Attribute("name", "String"));
    person.addAttribute(new Attribute("age", "Number"));
    Entity student = new Entity("Student");
    student.addAttribute(new Attribute("id", "Number"));
    Entity teacher = new Entity("Teacher");
    Entity course = new Entity("Course");
    course.addAttribute(new Attribute("title", "String"));
    Relation r1 = new Relation("Inheritance", person, student);
    Relation r2 = new Relation("Inheritance", person, teacher);
    Relation r3 = new Relation("1-1", course, teacher);
    Relation r4 = new Relation("Many-Many", course, student);

    system = new EntitySystem("University");
    system.addEntity(person);
    system.addEntity(teacher);
    system.addEntity(student);
    system.addEntity(course);
    system.addRelation(r1);
    system.addRelation(r2);
    system.addRelation(r3);
    system.addRelation(r4);
}
```

Internal DSL Syntax

- Standard approaches to fluent interfaces
  - Method chaining
  - Function sequence
  - Nested function
  - Closures

Internal DSL Patterns

Internal DSL Patterns

Metamodel (Semantic Model)

Expression builder (fluent-API)

Context Variable (what we operate on)

Construction Builder (immutable objects)

Symbol table (names)

Class symbol table (static typing)

Internal DSL Patterns

Metamodel (Semantic Model)

Expression builder (fluent-API)

Context Variable (what we operate on)

Construction Builder (immutable objects)

Symbol table (names)

Class symbol table (static typing)

Internal DSL Patterns

Metamodel (Semantic Model)

Expression builder (fluent-API)

Context Variable (what we operate on)

Construction Builder (immutable objects)

Symbol table (names)

Class symbol table (static typing)

Internal DSL: outside Java

Internal DSL more examples

More *exotic* approaches: LISP, Python, Smalltalk, C...

- Literal Map (moveTo(x,y) or moveToX: x Y: y)
- Dynamic Reception (redefined *method not understood*)
- Literal Extensions (Seconds(5) or 5 seconds)
- Macros (preprocessing)...

Any ideas? Please, donate examples to the repo :)

Internal DSL more examples

Even more …

- Annotations
- Parse tree manipulation
- Textual Polishing
- Notifications
- …

Internal DSL & Metamodel

Metamodel

In our class

Metamodel

Model a solution for a *family* of problems

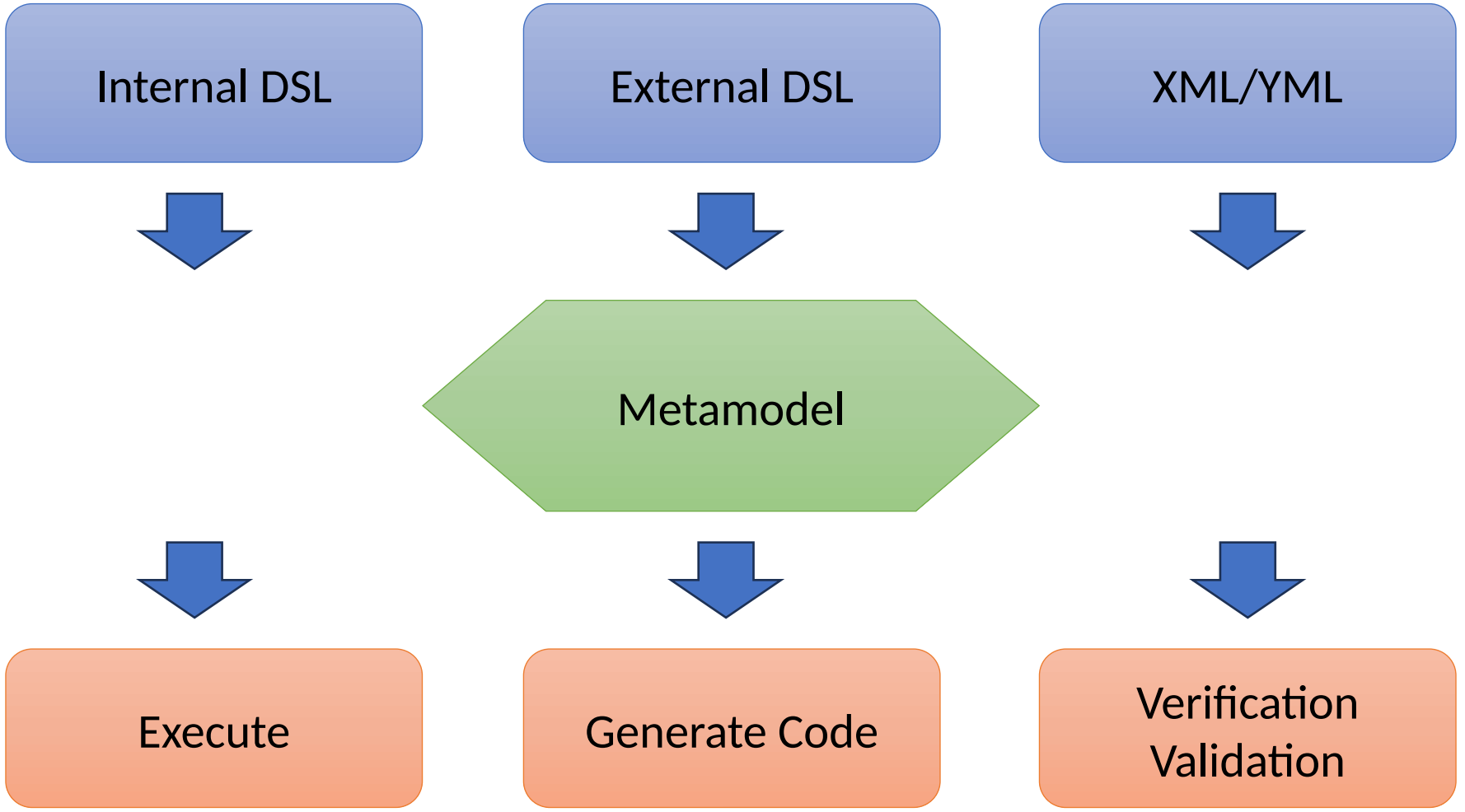
Model

Model a solution for *one* problem

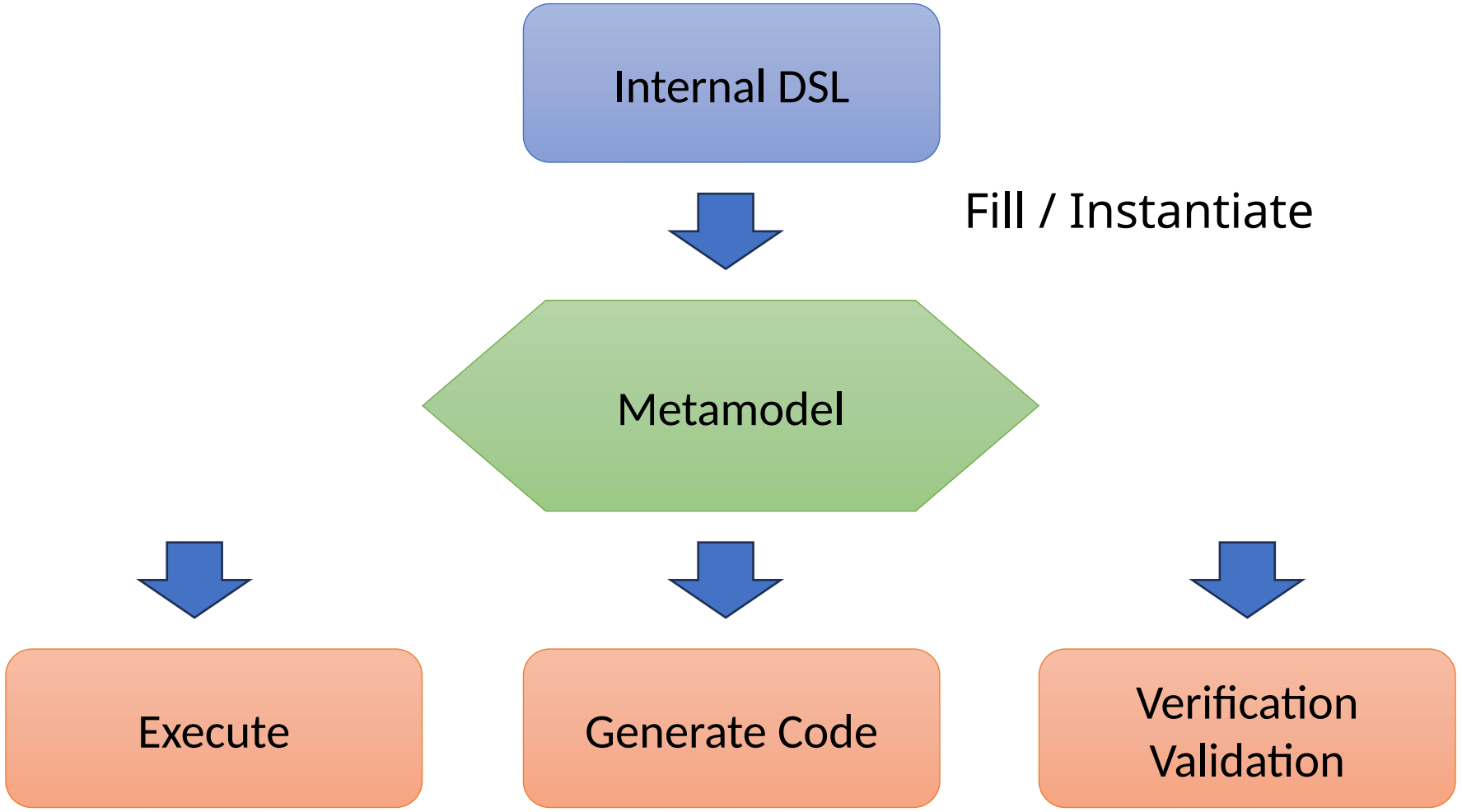
Metamodel

- In our class
  - Metamodel: Internal / External DSL
    - But: decoupling of Metamodel with DSL via parsing layer
    - We have *parsing* with internal DSL: Fluent-API
  - Model
    - Metamodel instance
    - A program written using a DSL

Metamodel


Internal DSL
External DSL
XML/YML
Metamodel
Execute
Generate Code
Verification
Validation

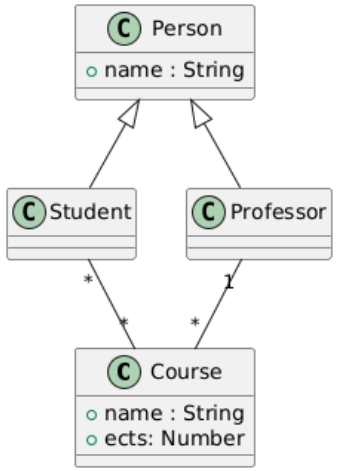
Internal DSL and metamodel

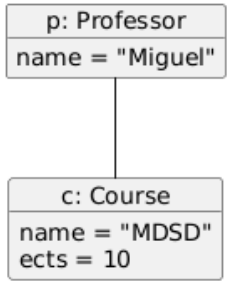


Class diagram metamodel / model

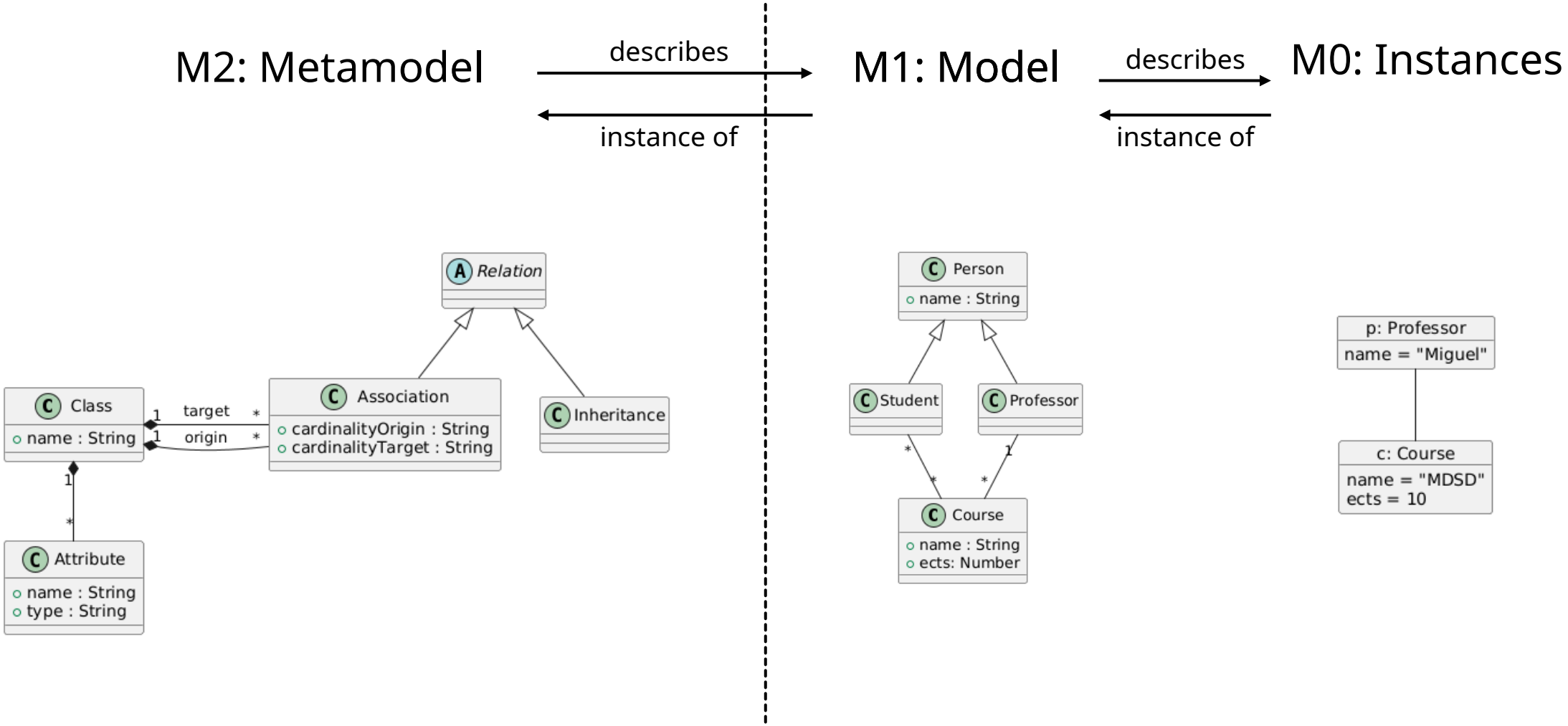
M1: Model describes M0: Instances

instance of





Class diagram metamodel / model



Group project

Group project: Internal DSL

- Syntax
  - On a programming language of your choice
    - Which one? Why? How does it support your *syntax (fluent API)*?
  - Code a representative example
  - Explain used techniques
- Implementation
  - Make your internal DSL *functional*
  - Being able to show that you are *executing* the metamodel
    - Miguel example: after build, text representation of a class diagram

Summary

Summary

Warm-up Internal DSL

Internal DSL Syntax

Internal DSL Patterns

```
public void build() {
    system("University").
        entity("Person").
            attribute("name", "String").
            attribute("age", "Number").
        entity("Student").
            attribute("id", "Number").
        entity("Teacher").
        relation("Inheritance", "Person", "Student").
        relation("Inheritance", "Person", "Teacher");
}
```

```
#lang racket
(entitysystem "University"
              (entity "Person"
                      (attribute "name" "String")
                      (attribute "age" "Number"))
              (entity "Student"
                      (attribute "id" "Number"))
              (entity "Teacher")
              (relation "Inheritance" "Person" "Student"))
```